

Notes on Chapter 12: Logical Vectors

Norman Lim

2025-07-24

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(nycflights13)
```

Showing R's vectorized operation:

```
x <- c(1, 2, 3, 5, 7, 11, 13)
x*2
```

```
## [1]  2  4  6 10 14 22 26
```

- Any manipulation we can do to a free-floating vector, we can do to a variable inside a data frame using `mutate()`:

```
df <- tibble(x)
df |>
  mutate(y = x * 2)
```

```
## # A tibble: 7 x 2
##       x     y
##   <dbl> <dbl>
## 1     1     2
## 2     2     4
## 3     3     6
## 4     5    10
## 5     7    14
## 6    11    22
## 7    13    26
```

Comparisons

Using logical conditions:

```
flights |>
  filter(dep_time > 660 & dep_time < 2000 & abs(arr_delay) < 20)
```

```
## # A tibble: 156,253 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     711             715          -4    1151          1206
## 2  2013     1     1     712             715          -3    1023          1035
## 3  2013     1     1     717             720          -3     850           840
## 4  2013     1     1     719             721          -2    1017          1012
## 5  2013     1     1     723             725          -2    1013          1017
## 6  2013     1     1     724             725          -1    1020          1030
## 7  2013     1     1     725             730          -5    1052          1040
## 8  2013     1     1     727             730          -3     959           952
## 9  2013     1     1     728             732          -4    1041          1038
## 10 2013     1     1     732             735          -3     857           858
## # i 156,243 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Instad of doing it like this, mutate() provides a cleaner way to filter and create a new variable:

```
flights |>
  mutate(
    daytime = dep_time > 600 & dep_time < 2000,
    approx_ontime = abs(arr_delay) < 20,
    .keep = "used"
  )
```

```
## # A tibble: 336,776 x 4
##   dep_time arr_delay daytime approx_ontime
##   <int>     <dbl> <lgl>    <lgl>
## 1     517        11 FALSE    TRUE
## 2     533        20 FALSE   FALSE
## 3     542        33 FALSE   FALSE
## 4     544       -18 FALSE    TRUE
## 5     554       -25 FALSE   FALSE
## 6     554        12 FALSE    TRUE
## 7     555        19 FALSE    TRUE
## 8     557       -14 FALSE    TRUE
## 9     557        -8 FALSE    TRUE
## 10    558         8 FALSE    TRUE
## # i 336,766 more rows
```

- This cleaner way makes it easier to both read the code and check if each step has been computed correctly (as intended).

Compare the code above with:

```
flights |>
  mutate(
    daytime = dep_time > 600 & dep_time < 2000,
    approx_ontime = abs(arr_delay) < 20
  ) |>
  filter(daytime & approx_ontime)
```

```
## # A tibble: 172,286 x 21
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     601             600           1     844             850
## 2  2013     1     1     602             610          -8     812             820
## 3  2013     1     1     602             605          -3     821             805
## 4  2013     1     1     606             610          -4     858             910
## 5  2013     1     1     606             610          -4     837             845
## 6  2013     1     1     607             607           0     858             915
## 7  2013     1     1     611             600          11     945             931
## 8  2013     1     1     613             610           3     925             921
## 9  2013     1     1     615             615           0     833             842
## 10 2013     1     1     622             630          -8    1017            1014
## # i 172,276 more rows
## # i 13 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>, daytime <lgl>,
## #   approx_ontime <lgl>
```

Floating-point comparison

- Beware of using `==` when comparing numbers. Example:

```
x <- c(1/49 * 49, sqrt(2)^2)
x
```

```
## [1] 1 2
```

All seems OK until you test them for equality:

```
x == c(1, 2)
```

```
## [1] FALSE FALSE
```

Reason: The numbers contained in `x` are not exactly 1 and 2, respectively – they are slightly off because there is no way to exactly represent $1/49$ or $\sqrt{2}$. We can see the exact values stored in `x` like this:

```
print(x, digits = 16)
```

```
## [1] 0.9999999999999999 2.0000000000000004
```

The `near` function can be used in this case – it ignores small differences:

```
near(x, c(1, 2))
```

```
## [1] TRUE TRUE
```

Missing values

- Missing values are “contagious” – almost any operation involving an unknown will also be unknown:

```
NA > 5
```

```
## [1] NA
```

```
10 == NA
```

```
## [1] NA
```

```
NA == NA
```

```
## [1] NA
```

- For this reason, we cannot filter for NAs using `== NA`.

This will not work:

```
flights |>
  filter(dep_time == NA)
```

```
## # A tibble: 0 x 19
## # i 19 variables: year <int>, month <int>, day <int>, dep_time <int>,
## #   sched_dep_time <int>, dep_delay <dbl>, arr_time <int>,
## #   sched_arr_time <int>, arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- This is the reason why we have the `is.na()` tool.

Using `is.na()`

```
flights |>
  filter(is.na(dep_time))
```

```
## # A tibble: 8,255 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
## 1  2013     1     1     NA             1630         NA       NA             1815
## 2  2013     1     1     NA             1935         NA       NA             2240
## 3  2013     1     1     NA             1500         NA       NA             1825
## 4  2013     1     1     NA              600         NA       NA              901
## 5  2013     1     2     NA             1540         NA       NA             1747
## 6  2013     1     2     NA             1620         NA       NA             1746
## 7  2013     1     2     NA             1355         NA       NA             1459
## 8  2013     1     2     NA             1420         NA       NA             1644
## 9  2013     1     2     NA             1321         NA       NA             1536
## 10 2013     1     2     NA             1545         NA       NA             1910
## # i 8,245 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Sorting the results with `arrange()`:

```
flights |>
  filter(month == 1, day == 1) |>
  arrange(dep_time)
```

```
## # A tibble: 842 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
## 1  2013     1     1     517             515         2       830             819
## 2  2013     1     1     533             529         4       850             830
## 3  2013     1     1     542             540         2       923             850
## 4  2013     1     1     544             545        -1      1004            1022
## 5  2013     1     1     554             600        -6       812             837
## 6  2013     1     1     554             558        -4       740             728
## 7  2013     1     1     555             600        -5       913             854
## 8  2013     1     1     557             600        -3       709             723
## 9  2013     1     1     557             600        -3       838             846
## 10 2013     1     1     558             600        -2       753             745
```

```
## # i 832 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- By default, `arrange()` puts all the NAs at the end. We can override this by arranging the results in descending order after filtering:

```
flights |>
  filter(month == 1, day == 1) |>
  arrange(desc(is.na(dep_time)), dep_time)
```

```
## # A tibble: 842 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     NA           1630           NA       NA           1815
## 2  2013     1     1     NA           1935           NA       NA           2240
## 3  2013     1     1     NA           1500           NA       NA           1825
## 4  2013     1     1     NA            600           NA       NA            901
## 5  2013     1     1    517            515            2       830            819
## 6  2013     1     1    533            529            4       850            830
## 7  2013     1     1    542            540            2       923            850
## 8  2013     1     1    544            545           -1      1004           1022
## 9  2013     1     1    554            600           -6       812            837
## 10 2013     1     1    554            558           -4       740            728
## # i 832 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Exercises

1. How does `dplyr::near()` work? Type `near` to see the source code. Is `sqrt(2)^2` near 2?

Ans. `near` works by checking if the difference between the two values is less than the tolerance. If yes, then it is “near”.

```
near(sqrt(2)^2, 2)
```

```
## [1] TRUE
```

2. Use `mutate()`, `is.na()`, and `count()` together to describe how the missing values in `dep_time`, `sched_dep_time` and `dep_delay` are connected.

```
# test
```

Boolean Algebra

- Don’t use `&&` and `||` operators in `dplyr` functions. These are called short-circuiting operators. They are important for programming, not for data science.

Order of operations

Instruction: “Find all flights that departed in November or December.”

Wrong way of executing the instruction:

```
flights |>
  filter(month == 11 | 12)
```

```
## # A tibble: 336,776 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
## 1  2013     1     1     517           515         2     830           819
## 2  2013     1     1     533           529         4     850           830
## 3  2013     1     1     542           540         2     923           850
## 4  2013     1     1     544           545        -1    1004          1022
## 5  2013     1     1     554           600        -6     812           837
## 6  2013     1     1     554           558        -4     740           728
## 7  2013     1     1     555           600        -5     913           854
## 8  2013     1     1     557           600        -3     709           723
## 9  2013     1     1     557           600        -3     838           846
## 10 2013     1     1     558           600        -2     753           745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Correct way of executing the instruction:

```
flights |>
  filter(month == 11 | month == 12)
```

```
## # A tibble: 55,403 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
## 1  2013    11     1      5           2359         6     352           345
## 2  2013    11     1     35           2250        105     123          2356
## 3  2013    11     1    455           500         -5     641           651
## 4  2013    11     1    539           545         -6     856           827
## 5  2013    11     1    542           545         -3     831           855
## 6  2013    11     1    549           600        -11     912           923
## 7  2013    11     1    550           600        -10     705           659
## 8  2013    11     1    554           600         -6     659           701
## 9  2013    11     1    554           600         -6     826           827
## 10 2013    11     1    554           600         -6     749           751
## # i 55,393 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- This mistake can easily happen because of the way we say it in our everyday language.

`%in%`

- Instead of using `filter(x == y | x == z )` format (which is error-prone), we can simply use `%in%`.

```
flights |>
  filter(month %in% c(11, 12))
```

```
## # A tibble: 55,403 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
## 1  2013    11     1      5           2359         6     352           345
## 2  2013    11     1     35           2250        105     123          2356
## 3  2013    11     1    455           500         -5     641           651
## 4  2013    11     1    539           545         -6     856           827
```

```
## 5 2013 11 1 542 545 -3 831 855
## 6 2013 11 1 549 600 -11 912 923
## 7 2013 11 1 550 600 -10 705 659
## 8 2013 11 1 554 600 -6 659 701
## 9 2013 11 1 554 600 -6 826 827
## 10 2013 11 1 554 600 -6 749 751
## # i 55,393 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- The `%in%` avoids the issue we had with `==` when used to check for NAs:

```
# using ==
c(1, 2, NA) == NA
```

```
## [1] NA NA NA
```

```
c(1, 2, NA) %in% NA
```

```
## [1] FALSE FALSE TRUE
```

- The `%in%` expression can be used to filter for NAs:

```
flights |>
  filter(dep_time %in% c(NA, 0800))
```

```
## # A tibble: 8,803 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1 2013     1     1     800           800           0    1022          1014
## 2 2013     1     1     800           810          -10     949           955
## 3 2013     1     1      NA          1630           NA      NA           1815
## 4 2013     1     1      NA          1935           NA      NA           2240
## 5 2013     1     1      NA          1500           NA      NA           1825
## 6 2013     1     1      NA           600           NA      NA           901
## 7 2013     1     2     800           810          -10    1102          1116
## 8 2013     1     2      NA          1540           NA      NA           1747
## 9 2013     1     2      NA          1620           NA      NA           1746
## 10 2013     1     2      NA          1355           NA      NA           1459
## # i 8,793 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Exercises

1. Find all flights where `arr_delay` is missing but `dep_delay` is not. Find all flights where neither `arr_time` nor `sched_arr_time` are missing, but `arr_delay` is.

Ans.

```
flights |>
  filter(is.na(arr_delay) & !is.na(dep_delay))
```

```
## # A tibble: 1,175 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1 2013     1     1    1525          1530          -5    1934          1805
```

```
## 2 2013 1 1 1528 1459 29 2002 1647
## 3 2013 1 1 1740 1745 -5 2158 2020
## 4 2013 1 1 1807 1738 29 2251 2103
## 5 2013 1 1 1939 1840 59 29 2151
## 6 2013 1 1 1952 1930 22 2358 2207
## 7 2013 1 1 2016 1930 46 NA 2220
## 8 2013 1 2 905 822 43 1313 1045
## 9 2013 1 2 1125 925 120 1445 1146
## 10 2013 1 2 1848 1840 8 2333 2151
## # i 1,165 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
flights |>
  filter(!is.na(arr_time) & !is.na(arr_time) & is.na(arr_delay))
```

```
## # A tibble: 717 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1 2013     1     1    1525           1530          -5    1934           1805
## 2 2013     1     1    1528           1459           29    2002           1647
## 3 2013     1     1    1740           1745          -5    2158           2020
## 4 2013     1     1    1807           1738           29    2251           2103
## 5 2013     1     1    1939           1840           59     29           2151
## 6 2013     1     1    1952           1930           22    2358           2207
## 7 2013     1     2     905            822           43    1313           1045
## 8 2013     1     2    1125            925          120    1445           1146
## 9 2013     1     2    1848            1840            8    2333           2151
## 10 2013     1     2    1849            1724           85    2235           1938
## # i 707 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

2. How many flights have a missing `dep_time`? What other variables are missing in these rows? What might these rows represent?

Ans.

```
flights |>
  filter(is.na(dep_time))
```

```
## # A tibble: 8,255 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1 2013     1     1     NA           1630           NA     NA           1815
## 2 2013     1     1     NA           1935           NA     NA           2240
## 3 2013     1     1     NA           1500           NA     NA           1825
## 4 2013     1     1     NA           600           NA     NA           901
## 5 2013     1     2     NA           1540           NA     NA           1747
## 6 2013     1     2     NA           1620           NA     NA           1746
## 7 2013     1     2     NA           1355           NA     NA           1459
## 8 2013     1     2     NA           1420           NA     NA           1644
## 9 2013     1     2     NA           1321           NA     NA           1536
## 10 2013     1     2     NA           1545           NA     NA           1910
```



```
## # i 8,245 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>

n_missing_dep_time <- flights |>
  filter(is.na(dep_time)) |>
  nrow()
```

There are 8255 flights with missing `dep_time`. `dep_delay` and `arr_time` are also missing, along with missing `dep_time`.

3. Assuming that a missing `dep_time` implies that a flight is cancelled, look at the number of cancelled flights per day. Is there a pattern? Is there a connection between the proportion of cancelled flights and the average delay of non-cancelled flights?

Summaries

Logical summaries

- Using `all()` and `any()`:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    all_delayed = all(dep_delay <= 60, na.rm = TRUE),
    any_long_delay = any(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 365 x 5
##   year month   day all_delayed any_long_delay
##   <int> <int> <int> <lgl>         <lgl>
## 1  2013     1     1 FALSE         TRUE
## 2  2013     1     2 FALSE         TRUE
## 3  2013     1     3 FALSE         FALSE
## 4  2013     1     4 FALSE         FALSE
## 5  2013     1     5 FALSE         TRUE
## 6  2013     1     6 FALSE         FALSE
## 7  2013     1     7 FALSE         TRUE
## 8  2013     1     8 FALSE         FALSE
## 9  2013     1     9 FALSE         TRUE
## 10 2013     1    10 FALSE         TRUE
## # i 355 more rows
```

Numerical summaries

```
flights |>
  group_by(year, month, day) |>
  summarize(
    proportion_delayed = mean(dep_delay <= 60, na.rm = TRUE),
    count_long_delay = sum(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 365 x 5
##   year month   day proportion_delayed count_long_delay
```

```
##      <int> <int> <int>          <dbl>          <int>
## 1  2013      1      1          0.939            3
## 2  2013      1      2          0.914            3
## 3  2013      1      3          0.941            0
## 4  2013      1      4          0.953            0
## 5  2013      1      5          0.964            1
## 6  2013      1      6          0.959            0
## 7  2013      1      7          0.956            1
## 8  2013      1      8          0.975            0
## 9  2013      1      9          0.986            1
## 10 2013      1     10          0.977            2
## # i 355 more rows
```

Logical subsetting

Condition: find the average delay just for the flights that were actually delayed.

Plan of attack: first, filter for the delayed flights. Then, calculate the average delay.

```
flights |>
  filter(arr_delay > 0) |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay),
    n = n(),
    .groups = "drop"
  )
```

```
## # A tibble: 365 x 5
##   year month   day behind     n
##   <int> <int> <int>   <dbl> <int>
## 1  2013     1     1    32.5   461
## 2  2013     1     2    32.0   535
## 3  2013     1     3    27.7   460
## 4  2013     1     4    28.3   297
## 5  2013     1     5    22.6   238
## 6  2013     1     6    24.4   381
## 7  2013     1     7    27.8   243
## 8  2013     1     8    20.8   275
## 9  2013     1     9    25.6   287
## 10 2013     1    10    27.3   220
## # i 355 more rows
```

Condition: In addition to the conditions above, also compute for the average delay for flights that arrived early.

Plan of attack: similar to the plan above, but perform a separate filter step. Then figure out how to combine the two dataframes altogether.

Better plan of attack: Perform an inline filter: `arr_delay[arr_delay > 0]` to get only the positive arrival delays. This is similar to how boolean filtering is usually performed in Python.

```
flights |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
    ahead = mean(arr_delay[arr_delay < 0], na.rm = TRUE),
```

```

n = n(),
  .groups = "drop"
)

```

```

## # A tibble: 365 x 6
##   year month   day behind ahead     n
##   <int> <int> <int>   <dbl> <dbl> <int>
## 1  2013     1     1    32.5  -12.5   842
## 2  2013     1     2    32.0  -14.3   943
## 3  2013     1     3    27.7  -18.2   914
## 4  2013     1     4    28.3  -17.0   915
## 5  2013     1     5    22.6  -14.0   720
## 6  2013     1     6    24.4  -13.6   832
## 7  2013     1     7    27.8  -17.0   933
## 8  2013     1     8    20.8  -14.3   899
## 9  2013     1     9    25.6  -13.0   902
## 10 2013     1    10    27.3  -16.4   932
## # i 355 more rows

```

Exercises

1. What will `sum(is.na(x))` tell you? How about `mean(is.na(x))`
2. What does `prod()` return when applied to a logical vector? What logical summary function is it equivalent to? What does `min()` return when returned to a logical vector? What logical summary is it equivalent to?

Ans.

```

TT_vector <- c(TRUE, TRUE)
TF_vector <- c(TRUE, FALSE)
TFT_vector <- c(TRUE, FALSE, TRUE)
FTF_vector <- c(FALSE, TRUE, FALSE)

```

```
prod(TT_vector)
```

```
## [1] 1
```

```
prod(TF_vector)
```

```
## [1] 0
```

```
prod(TFT_vector)
```

```
## [1] 0
```

```
prod(FTF_vector)
```

```
## [1] 0
```

```
min(TT_vector)
```

```
## [1] 1
```

```
min(TF_vector)
```

```
## [1] 0
```

```
min(TFT_vector)
```

```
## [1] 0
```

```
min(FTF_vector)
```

```
## [1] 0
```

```
sum(all(TT_vector))
```

```
## [1] 1
```

```
sum(all(TF_vector))
```

```
## [1] 0
```

```
sum(all(TFT_vector))
```

```
## [1] 0
```

```
sum(all(FTF_vector))
```

```
## [1] 0
```

When applied to logical vectors, they act like `sum(all())`

Conditional transformations

Example using `if_else()`

```
x <- c(-3:3, NA)
```

```
if_else(x > 0, "+ve", "-ve")
```

```
## [1] "-ve" "-ve" "-ve" "-ve" "+ve" "+ve" "+ve" NA
```

The syntax is similar to the IF formula in Excel, but there is an optional argument specifies what to return instead of NA:

```
if_else(x > 0, "+ve", "-ve", "???)
```

```
## [1] "-ve" "-ve" "-ve" "-ve" "+ve" "+ve" "+ve" "???"
```

A simple version of the `coalesce()` function using `if_else()`

```
x1 <- c(NA, 1, 2, NA)
```

```
y1 <- c(3, NA, 4, 6)
```

```
ifelse(is.na(x1), y1, x1)
```

```
## [1] 3 1 2 6
```

- Just like in Excel, the `if_else()` function can be hard to read (and hard to code) when the conditions get complicated:

```
if_else(x == 0, "0", if_else(x < 0, "+-ve", "+ve"), "???)
```

```
## [1] "+-ve" "+-ve" "+-ve" "0" "+ve" "+ve" "+ve" "???"
```

- For complicated conditions, it is better to use `case_when()` instead of `if_else()`:

```
x <- c(-3:3, NA)
```

```
case_when(  
  x == 0 ~ "0",  
  x < 0 ~ "-ve",  
  x > 0 ~ "+ve",  
)
```

```
is.na(x) ~ "???"
)

## [1] "-ve" "-ve" "-ve" "0"      "+ve" "+ve" "+ve" "???"
```

The code is longer but it more readable and easier to code.

- With `case_when()`, it is also possible to create a default/catch-all value to return:

```
case_when(
  x < 0 ~ "-ve",
  x > 0 ~ "+ve",
  .default = "???"
)

## [1] "-ve" "-ve" "-ve" "???" "+ve" "+ve" "+ve" "???"
```

Just like `vlookup` in Excel, if multiple conditions match, only the first case will be used:

```
case_when(
  x > 0 ~ "+ve",
  x > 2 ~ "big"
)

## [1] NA      NA      NA      NA      "+ve" "+ve" "+ve" NA
```

- With `case_when()`, other logical conditions can be used:

```
flights |>
  mutate(
    status = case_when(
      is.na(arr_delay) ~ "cancelled",
      arr_delay < -30 ~ "very early",
      arr_delay < -15 ~ "early",
      abs(arr_delay) <= 15 ~ "on time",
      arr_delay < 60 ~ "late",
      arr_delay < Inf ~ "very late"
    ),
    .keep = "used"
  )
```

```
## # A tibble: 336,776 x 2
##   arr_delay status
##   <dbl> <chr>
## 1      11 on time
## 2      20 late
## 3      33 late
## 4     -18 early
## 5     -25 early
## 6      12 on time
## 7      19 late
## 8     -14 on time
## 9      -8 on time
## 10      8 on time
## # i 336,766 more rows
```

Exercises

1. A number is even if it's divisible by two, which in R you can find out with `x %% 2 == 0`. Use this fact and `if_else()` to determine whether each number between 0 and 20 is even or odd.

Ans.

```
x <- c(0:20)

if_else(x %% 2 == 0, "even", "odd")

## [1] "even" "odd"  "even" "odd"  "even" "odd"  "even" "odd"  "even" "odd"
## [11] "even" "odd"  "even" "odd"  "even" "odd"  "even" "odd"  "even" "odd"
## [21] "even"
```

2. Given a vector of days like `x <- c("Monday", "Saturday", "Wednesday")`, use `anif_else()` statement to label them as weekends or weekdays.

```
x <- c("Monday", "Saturday", "Wednesday", "March")
weekends <- c("Saturday", "Sunday")
days <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")

cond1 <- x %in% days
cond2 <- x %in% weekends
if_else(cond1, if_else(cond2, "weekends", "weekdays"), "???)

## [1] "weekdays" "weekends" "weekdays" "???"
```

3. Use `if_else()` to compute the absolute value of a numeric vector called `x`.

Ans.

```
x <- c(-3:3, NA)
if_else(x >= 0, x, -x)

## [1] 3 2 1 0 1 2 3 NA
```

4. Write a `case_when()` statement that uses the `month` and `day` columns from `flights` to label a selection of important US holidays (e.g., New Years Day, 4th of July, Thanksgiving, and Christmas). First create a logical column that is either `TRUE` or `FALSE`, and then create a character column that either gives the name of the holiday or is `NA`.

```
flights |>
  mutate(
    is_holiday = case_when(
      (day == 1 & month == 1) ~ TRUE,
      (day == 4 & month == 7) ~ TRUE,
      (day == 25 & month == 12) ~ TRUE,
      TRUE ~ NA
    ),
    holiday_type = case_when(
      (day == 1 & month == 1) ~ "New Years Day",
      (day == 4 & month == 7) ~ "Independence Day",
      (day == 25 & month == 12) ~ "Christmas Day"
    ),
    .keep = "used"
  )

## # A tibble: 336,776 x 4
##   month   day is_holiday holiday_type
```

##	<int>	<int>	<lgl>	<chr>
## 1	1	1	TRUE	New Years Day
## 2	1	1	TRUE	New Years Day
## 3	1	1	TRUE	New Years Day
## 4	1	1	TRUE	New Years Day
## 5	1	1	TRUE	New Years Day
## 6	1	1	TRUE	New Years Day
## 7	1	1	TRUE	New Years Day
## 8	1	1	TRUE	New Years Day
## 9	1	1	TRUE	New Years Day
## 10	1	1	TRUE	New Years Day

i 336,766 more rows